

# Scenario: Transaction Viewer Application

Develop a simple full-stack application for a hypothetical credit card company. The application should allow customers to view their recent transactions and allow the backend system to manage transaction states.

## Frontend Requirements (Customer View):

- **Transaction List:** Display a list of transactions (date, merchant name, amount, status).
- **Status Filter:** Implement a basic filter to view transactions by status (e.g., Posted, Pending, Reversed).
- **Simple UI:** A clean, responsive design (framework choice is up to the candidate, e.g., React, Vue, Angular).

## Backend Requirements (API Endpoints):

The API should expose the following functionalities:

- **Retrieve Transactions:** An endpoint to fetch all transactions for a single user/account.
- **Create Transaction:** An endpoint to create a new transaction (simulating a purchase).
- **Reverse Transaction:** An endpoint to update an existing transaction's status to 'Reversed'.
  - *Logic:* Only transactions with a 'Posted' status can be reversed.

## Constraints & Expectations:

| Component               | Expectation                                                                                                                                                                                                                          |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>AI Usage</b>         | Explicitly allowed and encouraged. The candidate must briefly describe <i>how</i> they utilized AI in a README section (e.g., "Used AI for initial React component scaffolding," "Used AI to generate database schema boilerplate"). |
| <b>Technology Stack</b> | Candidate's choice (e.g., Node/Express + React, Python/Flask + Vue, etc.). Must be a modern (reasonable) standard stack.                                                                                                             |
| <b>Persistence</b>      | Use a simple file (e.g., JSON file) for data persistence. <b>A complex database setup (e.g., PostgreSQL, MongoDB) is NOT required.</b>                                                                                               |
| <b>Code Quality</b>     | Clean, well-structured, readable code with appropriate separation of concerns (API routes separate from business logic).                                                                                                             |

| Component            | Expectation                                                                                                                                                                                                |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Documentation</b> | A clear <b>README.md</b> file with setup instructions (including necessary dependencies), run instructions, and the AI usage summary - focusing on prompts and results.<br><b>PROMPTS.md</b> is preferred! |

## Notes

Do not leave your repo public.

And send us the git bundle (see <https://git-scm.com/docs/git-bundle>)